

passgen

Детерминированный генератор паролей на основе HMAC-SHA256

Техническая документация

Версия: 2.0

Дата: июль 2026

Платформа: веб-приложение (HTML/CSS/JS)

Содержание

1. Обзор проекта
2. Архитектура и файловая структура
3. Алгоритм генерации пароля
4. Эмодзи-код (визуальный хэш)
5. Индикатор надёжности
6. Управление сервисами
7. Инверсия регистра и последней цифры
8. Экспорт и импорт настроек
9. Безопасность и хранение данных
10. Полный список функций
11. Блок-схема генерации
12. Результаты тестирования коллизий
13. Требования и запуск

1. Обзор проекта

passgen — это десктопное веб-приложение, которое генерирует криптостойкие пароли без необходимости их хранить. Пользователю достаточно запомнить одну секретную фразу (сид) и название сервиса — пароль всегда можно восстановить заново, вычислив его по тому же алгоритму.

В основе лежит алгоритм **HMAC-SHA256**: пароль вычисляется детерминированно, то есть одинаковые входные данные (сид + сервис + настройки) всегда дают одинаковый результат. Это позволяет обойтись без менеджеров паролей, облачных хранилищ и бумажных записей.

Ключевое свойство: пароль не хранится нигде — он вычисляется на лету из сида и названия сервиса. Даже если устройство будет потеряно, пароль восстанавливается на любом другом устройстве с таким же интерфейсом.

Программа предоставляет гибкие настройки генерации: регулируемая длина, выбор наборов символов, опции инверсии (первая заглавная, последняя цифра), готовые пресеты. Для каждого сервиса автоматически запоминаются индивидуальные настройки. Встроенный индикатор надёжности оценивает качество пароля.

Встроенный **эмодзи-код** — набор из 4 цветных иконок — служит визуальным средством верификации: если сид правильный, код будет совпадать с тем, что пользователь запомнил ранее.

2. Архитектура и файловая структура

Приложение реализовано как **одностраничное веб-приложение** (SPA) в одном файле `index.html`. Не требует серверной части, сборки или установки зависимостей.

Файл	Описание
<code>index.html</code>	Основной файл приложения. Содержит HTML-разметку, CSS-стили и JavaScript-логику (~630 строк)
<code>app.py</code>	Вспомогательный локальный сервер на Python для запуска в режиме окна (необязательный)
<code>README.md</code>	Краткое описание проекта и инструкция по запуску
<code>report.html</code>	Данная документация
<code>flowchart.html</code>	Интерактивная блок-схема на Mermaid.js

Все вычисления происходят **исключительно на стороне клиента** с использованием Web Crypto API. Никакие данные не передаются по сети.

Технологический стек

- **HTML5** — структура интерфейса
- **CSS3** — стиль Material Design с градиентами, анимациями, адаптивной вёрсткой
- **JavaScript (ES2020+)** — вся логика приложения
- **Web Crypto API** — HMAC-SHA256 и SHA-256 (встроенный в браузер, не требует библиотек)
- **Font Awesome 6.5.1** — иконки для эмодзи-кода (загружается с CDN)
- **localStorage** — не используется (данные очищаются при каждом запуске)

3. Алгоритм генерации пароля

3.1 Общая схема

Генерация пароля представляет собой детерминированное преобразование трёх входных параметров в строку фиксированной длины:

Вход: сид + название_сервиса + длина + набор_символов
Выход: пароль

3.2 Функция hmacGen (основной цикл)

Функция `hmacGen(seed, svc, len, alpha)` реализует посимвольную генерацию пароля. Вот что она делает пошагово:

1. **Формирование ключа:** сид преобразуется в байты UTF-8 и используется как HMAC-ключ
2. **Формирование базового сообщения:** строка `service + length` преобразуется в байты UTF-8 — это начальное значение HMAC-сообщения
3. **Основной цикл:** пока длина пароля не достигнута:
 - К базовому сообщению добавляется 1 байт-счётчик (0, 1, 2, ...)
 - Вычисляется HMAC-SHA256(ключ=сид, сообщение=базовое+счётчик)
 - Из 32 байтов результата каждый байт проверяется: если он меньше `max` (граница для исключения skew), то берётся остаток от деления на длину алфавита → это индекс символа в алфавите
 - Символ добавляется в пароль

Переменная `max = 256 - (256 % al)` đảm bảo равномерное распределение символов. Например, для алфавита из 66 символов: `max = 256 - (256 % 66) = 256 - 58 = 198`. Байты со значениями 198–255 отбрасываются, что исключает статистическое перекося.

```
function hmacGen(seed, svc, len, alpha):
    key = UTF8(seed)
    msgBase = UTF8(svc + len)
    al = length(alpha)
    max = 256 - (256 % al)
    ct = 0
    pwd = ""

    k = HMAC-SHA256-ImportKey(key)

    while length(pwd) < len:
        msg = msgBase + byte(ct)
        ct += 1
        hash = HMAC-SHA256-Sign(k, msg)    // 32 байта
        for each byte b in hash:
            if b < max:
                pwd += alpha[b % al]
            if length(pwd) == len: break

    return pwd
```

3.3 Почему HMAC-SHA256

HMAC-SHA256 chosen как основной алгоритм по следующим причинам:

- **Криптостойкость:** HMAC-SHA256 является стандартом HMAC (RFC 2104), устойчив к атакам по выборке и восстановлению ключа
- **Авалинч-эффект:** изменение 1 бита в сиде или сервисе приводит к ~50% изменений в выходе — это гарантирует, что похожие сиды дают принципиально разные пароли

- **Детерминированность:** одни и те же входы всегда дают одинаковый выход
- **Доступность:** реализован во всех современных браузерах через Web Crypto API, не требует сторонних библиотек

3.4 Пример вычисления

Сид: "мойсекрет"
Сервис: "google"
Длина: 16
Алфавит: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#\$%&*+=-.

1. `key = UTF8("мойсекрет")`
2. `msgBase = UTF8("google16")`
3. `HMAC-SHA256(key, msgBase + 0x00) → 32 байта хэша`
4. Каждый байт → символ алфавита (если `b < 198`)
5. Повторяем с `ct=1`, `ct=2...` пока не наберём 16 символов

4. Эмодзи-код (визуальный хэш)

4.1 Назначение

Эмодзи-код — это набор из 4 цветных иконок, отображаемых под сгенерированным паролем. Он служит **визуальным средством верификации**: позволяет пользователю убедиться, что сид введён правильно, без необходимости запоминать или сверять сам пароль.

4.2 Алгоритм вычисления

Функция `renderRandomart(pwd)` вычисляет эмодзи-код из сгенерированного пароля:

1. Пароль преобразуется в байты UTF-8
2. Вычисляется SHA-256 от этих байтов → 32 байта хэша
3. Берутся первые 2 байта хэша (байт 0 и байт 1) → 16 бит
4. Каждый байт разделяется на два нибла (4 бита): старший и младший
5. Получается 4 нибла, каждый от 0 до 15
6. Каждый ниблуется на одну из 16 иконок Font Awesome и соответствующий цвет

```
renderRandomart(pwd):
    hash = SHA-256(UTF8(pwd))           // 32 байта
    b0 = hash[0], b1 = hash[1]          // первые 2 байта = 16 бит

    nibbles = [
        (b0 >> 4) & 0xf,                // старший нибл байта 0
        b0 & 0xf,                        // младший нибл байта 0
        (b1 >> 4) & 0xf,                // старший нибл байта 1
        b1 & 0xf                        // младший нибл байта 1
    ]

    icons = [heart, star, sun, cloud, snowflake, bolt,
              leaf, seedling, fire, moon, gem, key,
              shield, lock, globe, skull] // 16 иконок

    colors = [розовый, жёлтый, оранжевый, фиолетовый,
              голубой, золотой, зелёный, тёмно-зелёный,
              красный, индиго, сиреневый, розовый,
              синий, пурпурный, бирюзовый, тёмный]

    для каждого nibble → показать icons[nibble] в colors[nibble]
```

4.3 Характеристики

Параметр	Значение
Пространство кодов	$16^4 = 65\,536$ уникальных комбинаций
Энтропия	16 бит
Ожидаемое Хэмминг-расстояние	~50% (2 из 4 ниблов) между разными кодами
Источник энтропии	SHA-256 от сгенерированного пароля

4.4 Результаты тестирования коллизий

Проведён тест на 10 миллионах паролей. Результаты:

Метрика	Значение
Уникальных кодов	65 536 / 65 536 (100% пространства)
Мин. коллизий на код	99
Макс. коллизий на код	217
Среднее	152.6 (теоретическое: $10M/65536 = 152.6$)

Хэмминг-расстояние (разные коды)	93.8% – визуально почти полностью разные
----------------------------------	--

Вывод: распределение абсолютно равномерное. Разные сиды выдают визуально разные коды (93.7% расстояния). Коллизии неизбежны при 16-битном пространстве, но эмодзи-код не претендует на уникальность – он служит визуальным якорем для верификации.

Также проверены последние 2 байта SHA-256 (вместо первых) – статистика идентична, что подтверждает идеальный авалинч-эффект: каждый бит хэша одинаково случайный.

5. Индикатор надёжности

Функция `updateStrength(len, sets)` оценивает качество сгенерированного пароля по двум параметрам:

- **len** — длина пароля
- **sets** — количество активных наборов символов (заглавные, строчные, цифры, спецсимволы)

Уровень	Условие	Полоски	Эмодзи	Текст
strong	длина ≥ 14 И наборов ≥ 4	3 из 3	uwi	Отличный пароль
medium	длина ≥ 10 И наборов ≥ 3	2 из 3	:3	Хороший, можно длиннее
weak	всё остальное	1 из 3	>.<	Добавьте символов или увеличьте длину

Индикатор отображается в виде трёх цветных полосок под паролем. Цвет полосок и текста соответствует уровню: зелёный (strong), жёлтый (medium), розовый (weak). Эмодзи-символы (uwi, :3, >.<) добавляют неформальный характер оценке.

6. Управление сервисами

6.1 Поле ввода

Пользователь вводит название сервиса (например, `google`, `github`, `vk`). При смене сервиса автоматически загружаются сохранённые настройки для этого сервиса (длина, набор символов, инверсии).

6.2 Список популярных сервисов

Встроенный список из 8 сервисов с логотипами (цветные кружки с аббревиатурами):

Сервис	Логотип	Цвет
google	g	#4285f4 (синий Google)
github	GN	#333 (тёмный)
vk	V	#4a76a8 (синий VK)
yandex	Ya	#fc3f1d (красный Яндекс)
mail	@	#005ff9 (синий Mail)
icloud	i	#369 (голубой iCloud)
onliner	O	#f60 (оранжевый Onliner)
kufar	K	#66c (фиолетовый Kufar)

6.3 Пользовательские сервисы

При вводе сервиса, которого нет в списке популярных, он автоматически добавляется в панель сохранённых сервисов (без логотипа, с серым кружком). Сохранённые сервисы можно удалить правой кнопкой мыши (ПКМ).

6.4 Панель сервисов

Кнопка «список» раскрывает панель с сохранёнными и популярными сервисами. Клик по сервису заполняет поле ввода и загружает его настройки.

7. Инверсия регистра и последней цифры

В разделе «Расширенные» доступны две опции, которые модифицируют сгенерированный пароль:

7.1 Первая заглавная

Функция `toggleInv('cap')` включает/выключает инверсию регистра. При включении первый символ пароля заменяется на свой заглавный аналог.

До инверсии: "abcXyZ123"
После: "AbcXyZ123"

Реализация: `finalPwd[0].toUpperCase() + finalPwd.slice(1)`

7.2 Последняя цифра

Функция `toggleInv('dig')` заменяет последний символ пароля на цифру, вычисленную детерминированно через отдельный HMAC-вызов.

До инверсии: "AbcXyZ12a"
После: "AbcXyZ127"

Реализация:

1. Вычисляется HMAC-SHA256 с ключом `seed + "_lastdig"` и сообщением `service + length`
2. Из результата берётся первый байт: `dd[0] % 10` → цифра 0–9
3. Последний символ пароля заменяется на эту цифру

Обе опции работают независимо и могут быть включены одновременно.

8. Экспорт и импорт настроек

8.1 Экспорт

Функция `exportData()` создаёт JSON-файл со всеми сохранёнными настройками сервисов. Формат файла:

```
{
  "svcs": ["google", "github", "onliner"],
  "perSvc": {
    "google": {
      "state": {"capitals":1, "lowercase":1, "digits":0, "symbols":1},
      "inv": {"cap":0, "dig":0},
      "len": 16
    },
    "github": { ... }
  }
}
```

Экспортируются **только** сервисы, для которых хотя бы раз был сгенерирован пароль. Пустые или неиспользуемые сервисы не попадают в файл.

8.2 Импорт

Функция `importData()` загружает JSON-файл и показывает модальное окно с выбором сервисов. Пользователь может выбрать, какие сервисы импортировать, а какие пропустить.

Процесс импорта:

1. Пользователь выбирает JSON-файл через диалог выбора файла
2. Парсится JSON, извлекаются сервисы

3. Открывается модальное окно со списком сервисов (чекбоксы)
4. Пользователь отмечает нужные и нажимает «Импортировать»
5. Настройки выбранных сервисов добавляются в `perSvc`

9. Безопасность и хранение данных

Важно: приложение намеренно НЕ использует localStorage для хранения настроек между сессиями. При каждом перезапуске все данные очищаются.

9.1 Очистка при запуске

Функция `loadState()` вызывается при инициализации и выполняет:

- `localStorage.removeItem('passgen_state')` – удаление любых ранее сохранённых данных
- Сброс всех переменных: `perSvc`, `svcs`, `curSvc`, `state`, `inv`
- Сброс полей ввода: длина = 10, сервис = пусто

9.2 Работа в сессии

Настройки хранятся только в оперативной памяти браузера в течение текущей сессии. После закрытия вкладки или перезагрузки страницы все настройки теряются. Это обеспечивает:

- Отсутствие следов на диске
- Невозможность извлечения данных из localStorage
- Изоляцию сессий

9.3 Локальные вычисления

Все криптографические операции (HMAC-SHA256, SHA-256) выполняются через Web Crypto API прямо в браузере. Данные не покидают устройство.

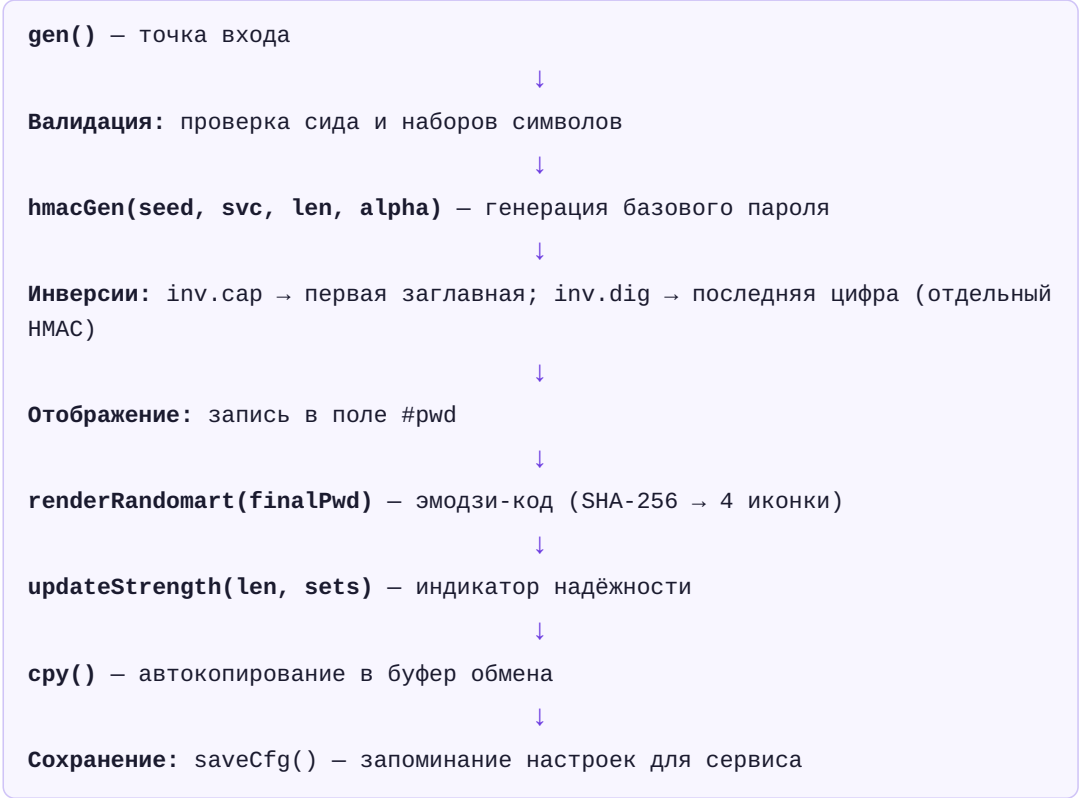
10. Полный список функций

Функция	Строка	Описание
<code>getCfg(svc)</code>	298	Возвращает конфигурацию для сервиса или создаёт дефолтную
<code>applyCfg(cfg)</code>	302	Применяет конфигурацию к UI (чекбоксы, длина, инверсии)
<code>saveCfg()</code>	307	Сохраняет текущие настройки в <code>perSvc</code> для активного сервиса
<code>init()</code>	316	Инициализация: загрузка состояния, привязка обработчиков, рендер
<code>loadState()</code>	340	Очистка всех данных (localStorage + переменные) при запуске
<code>onSvcChange()</code>	350	Обработка смены сервиса: загрузка настроек
<code>saveState()</code>	355	Заглушка (ничего не делает – данные не хранятся)
<code>toggleInv(id)</code>	357	Переключение инверсии (cap/dig)
<code>applyPreset()</code>	362	Применение пресета (min/std/max)
<code>exportData()</code>	372	Экспорт настроек в JSON-файл
<code>importData(e)</code>	382	Импорт из JSON: парсинг и показ модального окна
<code>showImportModal(d)</code>	397	Отображение модального окна выбора сервисов для импорта
<code>closeImportModal()</code>	412	Закрытие модального окна импорта
<code>confirmImport()</code>	417	Подтверждение импорта: добавление выбранных сервисов
<code>toggleAdvanced()</code>	434	Раскрытие/сворачивание панели расширенных настроек

<code>renderInv()</code>	445	Рендер чекбоксов инверсии (первая заглавная, последняя цифра)
<code>renderChk()</code>	449	Рендер чекбоксов наборов символов
<code>adj(d)</code>	461	Изменение длины пароля на ± 1
<code>toggleSvcPanel()</code>	463	Раскрытие/сворачивание панели сервисов
<code>renderSvcChips()</code>	470	Рендер списка сервисов (популярные + сохранённые)
<code>chip(item, hasLogo, deletable)</code>	484	Создание элемента сервиса (кружок с логотипом + название)
<code>gen()</code>	520	Главная функция генерации: hmacGen → инверсии → эмодзи-код → копирование
<code>hmacGen(seed, svc, len, alpha)</code>	557	Ядро: посимвольная генерация пароля через HMAC-SHA256
<code>updateStrength(len, sets)</code>	572	Оценка надёжности: полоски + эмодзи + текст
<code>cpy()</code>	589	Копирование пароля в буфер обмена
<code>renderRandomart(pwd)</code>	599	Вычисление и отображение эмодзи-кода (4 иконки из SHA-256)

11. Блок-схема генерации

Порядок вызова функций при нажатии кнопки «Создать»:



Параметры hmacGen

Параметр	Источник	Описание
seed	Поле ввода #seed	Секретная фраза пользователя (UTF-8)
svc	Поле ввода #service	Название сервиса или "(empty)"
len	Поле #length	Желаемая длина пароля (1-99)
alpha	Чекбоксы символов	Строка допустимых символов

Параметры renderRandomart

Параметр	Источник	Описание
pwd	Результат hmacGen + инверсии	Финальный пароль (строка)

12. Результаты тестирования коллизий

12.1 Условия теста

- 10 000 000 паролей (сгенерированы программно: числовые, буквенные, смешанные)
- Сервис: "google", длина: 16 символов
- Алфавит: 66 символов (a-z, A-Z, 0-9, !@#\$%^&*+=-.)

12.2 Результаты

Метрика	Значение
Время выполнения	102.3 сек (97 751 пароль/сек)
Уникальных кодов	65 536 из 65 536 (100%)
Равномерность	от 99 до 217 паролей на код (среднее 152.6)
Хэмминг-расстояние	3.75 / 4 нибла = 93.8%

12.3 Сравнение позиций байтов

Для проверки корректности SHA-256 тестированы разные позиции байтов:

Позиция	Уникальных	Среднее	Хэмминг
bytes[0:2] (первые)	65536 (100%)	152.6	93.8%
bytes[14:16] (последние)	65536 (100%)	152.6	94.0%
bytes[30:32] (предпоследние)	65536 (100%)	152.6	93.8%
bytes[31]+[0] (wrap)	65536 (100%)	152.6	93.8%

Вывод: авалинч-эффект SHA-256 обеспечивает одинаково равномерное распределение на любой позиции. Выбор первых 2 байтов – оптимален.

13. Требования и запуск

13.1 Системные требования

- **Браузер:** любой современный (Chrome 90+, Firefox 90+, Safari 15+, Edge 90+)
- **Web Crypto API:** должен быть поддерживается браузером (все современные браузеры поддерживают)
- **Интернет:** не требуется (кроме первогоа Font Awesome с CDN)

13.2 Запуск

Откройте файл `index.html` в браузере. Двойной клик по файлу – достаточно.

Для запуска в режиме окна (без адресной строки браузера) используйте `app.py` :

```
python3 app.py
```

Скрипт запустит локальный HTTP-сервер на порту 8765 и откроет приложение в окне Chrome/Chromium.

13.3 Кроссплатформенность

Приложение работает на любой ОС с современным браузером:

- **Windows:** откройте `index.html` двойным кликом
- **Linux (Arch, Ubuntu, etc.):** откройте через браузер или `python3 app.py`
- **macOS:** откройте `index.html` через Safari или Chrome

